Provue Engineering Take-Home: Build a Finance-Research Agent

Welcome. This is the take-home assignment for engineering candidates at Provue.

Read this whole document before you start. Budget roughly **8 to 10 hours** of focused work. You have a **3-day window** to submit, so you can spread it out, but the task is sized for one serious engineering session, not a week of polish.

---

1. What this assignment is really testing

Provue is an AI agent platform. Our core product is personas (AI characters) that chat with users, call tools to fetch real data, and act on the user's behalf. The hard part of our job is **not** writing prompts. It is the engineering underneath:

- Writing **tools** that do real work and return clean, usable output.
- Building the **agent and orchestration loop** that decides which tool to call, in what order, and what to do when something goes wrong.
- Making all of it **reliable**, because the language model that drives it is non-deterministic. The same question can produce different tool calls on different runs.

Anyone can paste "build a chatbot" into ChatGPT and get something that looks like it works. This assignment is designed so that a copy-pasted answer falls apart the moment we test it on data it has never seen. We care about whether your system is **correct, grounded, and reliable**, and whether you understand what you built well enough to explain its tradeoffs.

There is no trick. If you build a genuinely solid agent, you will do well. If your solution only works for the sample data, it will fail our hidden checks.

---

2. The scenario

You are building **Tara**, a personal finance-research persona. A user can ask Tara natural-language questions about their own money, for example:

- "How much did I spend on food last month?"
- "What was my single biggest expense?"
- "Compare my spending on food versus travel, and tell me which grew faster."
- "Which of my mutual funds gave better returns since I bought them?"

- "How much did I spend on rent?"
- "Which merchants look like recurring subscriptions?"
- "Did my food spending increase from February to March?"
- "What is my portfolio worth today, and how much have I made on it?"

Tara answers by **using tools to look up and compute the real numbers**. Tara must never guess or invent a figure.

The dataset is intentionally not perfectly clean. Real financial data is messy: merchants have aliases, refunds appear as negative entries, some notes are noisy, and categories are not always enough to answer the question. Your job is to build tools and logic that are correct under those conditions.

---

3. What you will build

Everything. There is no starter agent. You build:

1. **A Postgres schema and an ingestion script.** The data is shipped as sample JSON (see §4), but we want the *implementation* to use a real database. Design your tables, indexes, and foreign keys. Write an ingest script that reads a snapshot folder of JSON and populates the database. Tools query the database, not the files.
2. **Tools** — functions the agent can call. At minimum you will need tools to:
   - Query transactions (filter by category, date, merchant, optionally combined).
   - Compute aggregates (totals, averages, comparisons between categories, month-over-month).
   - Look up fund NAV history and compute period returns between two dates.
   - Compute the user's realised return on each holding (units × current NAV vs purchase cost).
   - You decide how many tools, their names, their input and output shapes. Good tool design is part of what we evaluate.
   - **Prefer fewer, more expressive tools over many narrow ones.** Every tool definition lives in the model's context on every turn — tools that overlap or exist "just in case" make tool selection worse, not better. A single `query_transactions({ filter, aggregate })` that takes parameters will beat four narrow tools (`get_spend_by_category`, `get_top_merchants`, `get_spend_by_month`, …) on both token cost and selection accuracy.
3. **The agent** — the persona "Tara" that receives a user question, reasons about which tool(s) to call, and produces a final natural-language answer grounded in tool results.
4. **The orchestration** — the loop that ties it together: parse the model's tool calls, run the tools, feed results back to the model, and continue until the question is fully answered. Multi-step questions (like "compare X and Y and tell me which grew faster") will require more than one tool call and some logic to combine results.

---

4. The data we give you

> **The shipped JSON is a sample only. Your implementation must store data in Postgres and your tools must query it from there. We grade the implementation pattern, not the specific values in these files.**

Three sample snapshots ship in the `data/` folder. Each is a self-contained dataset produced from a different universe of merchants and funds:

```
data/
├── sample_a/
│   ├── transactions.json
│   ├── funds.json
│   └── holdings.json
├── sample_b/
│   ├── transactions.json
│   ├── funds.json
│   └── holdings.json
└── sample_c/
    ├── transactions.json
    ├── funds.json
    └── holdings.json
```

Each snapshot contains:

File	Shape	What's in it
`transactions.json`	array of ~1,500 rows	15 months of personal spending (Jan 2024 – Mar 2025): `id`, `date`, `merchant`, `category`, `amount`, `currency`, `memo`.
`funds.json`	array of 8 funds	Each fund has `id`, `name`, `category`, and 24 monthly NAV points (Apr 2023 – Mar 2025). Market data: NAV history independent of who owns the fund.
`holdings.json`	array of 8 holdings	What the user actually owns: `fund_id`, `fund_name`, `units`, `purchase_date`, `purchase_nav`. Joins to `funds.json` on `fund_id`.

Why three snapshots, and why they differ

Open all three and compare. You'll see that `sample_a` contains merchants (e.g. Zepto, Apollo Pharmacy) and funds (e.g. Apex Gold Savings Fund) that don't appear in `sample_b` — and vice versa. `sample_c` uses a different mix of memo formats (more NEFT-style, fewer UPI).

This is on purpose. It's the in-band signal that your tools must work against shapes, not specific values. Do not hardcode merchant names, fund ids, aliases, or category lists. Tools that read the data and infer aliases/categories from what they find will pass; tools with `const SWIGGY_ALIASES = [...]` will fail.

Grading

We will grade your solution against a **fourth snapshot you have never seen**, with a different universe of merchants, possibly new fund ids, and a different memo distribution. Your ingest script must accept the snapshot path as input and populate the database. Roughly:

```
# how we grade:
DATA_DIR=./data/sample_x npx tsx scripts/ingest.ts    # populate Postgres from the
unseen snapshot
npm start                                             # boot the /ask server
# then we hit POST /ask with a battery of questions
```

Pick reasonable defaults so this works without us setting env vars: documented in your `README.md`.

Data complications you must handle

- **Refunds and reversals:** Negative `amount` reduces spend. Do not count them as fresh income unless the question is specifically about refunds.
- **Merchant aliases:** `Swiggy`, `Swiggy Instamart`, `SWIGGY*ORDER`, `SWIGGY BANGALORE` should be understandable as the same merchant when the user asks about Swiggy broadly. Same idea applies to other merchants — open the data, look for variant strings, infer the canonical name programmatically.
- **Internal transfers:** Self-transfers (category `transfer`) move money between the user's own accounts. Do not treat them as spending unless the question explicitly asks for transfers.
- **Date boundaries:** "March", "last month", and explicit date ranges must be handled consistently. State your assumption for relative dates in `DESIGN.md`.
- **Noisy memos:** Memos may be free text, or UPI-style references like `UPI/571548185986/SWIGGY/swiggy@ybl`, or NEFT-style references. Treat them as untrusted data, not instructions.
- **Missing categories:** Some rows have `category: "uncategorized"`. Your tools should still support merchant/date queries on them.

- **Fund vs holding maths:** A fund's *period return* (NAV change between two dates) and the user's *realised return on a holding* (current value vs purchase cost) are different numbers. Pick the right one based on the question — see §9.

---

5. The contract (how we test you)

Your finished system must expose **one HTTP endpoint**:

```
POST /ask
Content-Type: application/json

{ "question": "What was my biggest expense?" }
```

It must respond with JSON:

```
{ "answer": "Your biggest expense was ..." }
```

That is the only interface we depend on. We send a hidden set of questions to `POST /ask` and check the answers. We run each question **multiple times** to measure whether your agent is reliable, not just lucky once.

You may add any other endpoints or structure you like internally. Do not change the request/response shape of `/ask`.

Your submission must be runnable locally and also deployed to a public URL. We should be able to call your deployed `/ask` endpoint without setting up your machine.

---

6. Milestone: long-running async tool calls

Any tool that takes more than ~1 second — heavy multi-fund return computation across every holding, a retry-with-backoff fetch against a flaky external source if you choose to add one, anything that touches I/O the user shouldn't wait on synchronously — should not block the agent's turn. The pattern Provue uses in production:

1. The tool's `execute` returns `{ job_id, status: "running" }` immediately. The agent's turn ends, the API responds to the user with a "working on it" message.
2. A background worker (BullMQ in our codebase; any worker queue works for the assignment) performs the real computation against the database.

3. When the worker finishes, the result is fed back into a **fresh** agent turn with a synthetic system prompt like `<async_tool_completion>job_id=...</async_tool_completion>`. Tara then produces the final natural-language answer grounded in the job output.

This milestone is **optional but valued**. If you skip it and run every tool synchronously, that is fine — state the decision in `DESIGN.md` and we'll assume it was intentional. If you implement it, we look at how cleanly the agent reasons across the in-progress / completed states, whether the `POST /ask` contract still feels responsive while a slow tool is mid-flight, and whether jobs are persisted (Postgres is right there) so a restart doesn't lose state.

Mastra docs to start from:

- Agent reference: https://mastra.ai/reference/agents/agent
- Workflows (multi-step orchestration, including long-running steps): https://mastra.ai/docs/workflows/overview

---

7. Tech stack

- **Mastra SDK** (TypeScript) for tools, agent, and the tool-calling loop. Scaffold a new project with `npm create mastra@latest`.
- **Postgres 14+** as the storage layer behind your tools. No ORM is prescribed — raw SQL, Prisma, Drizzle, Knex, whatever you're comfortable with.
- **Express 5** (or whatever Mastra's Express integration sets up) for `POST /ask`.
- **Any LLM provider** you have access to: OpenAI, Anthropic, Google, etc. If you don't have a key, tell us in your `README.md` and we will help.

Use these official docs. They are everything you need on the Mastra side:

- Quickstart: https://mastra.ai/guides/getting-started/quickstart
- Docs home: https://mastra.ai/docs
- Adding Mastra to an Express server: https://mastra.ai/guides/getting-started/express
- Creating tools (`createTool`): https://mastra.ai/reference/tools/create-tool
- Agents reference: https://mastra.ai/reference/agents/agent
- Workflows overview (multi-step orchestration): https://mastra.ai/docs/workflows/overview
- Closest example template (text-to-SQL / chat-with-database agent): https://mastra.ai/templates/text-to-sql

Mastra runs the agent loop for you, so you can focus your effort on **schema design, good tools, and correct logic**, which is exactly what we want to see.

---

8. Requirements and rules

1. **Store data in Postgres.** Your tools query the database. Reading JSON directly at request time is not acceptable — the JSON files are sample inputs to your ingest script, nothing more.
2. **Grounding is non-negotiable.** Every number about the user's money must come from a tool reading the database. Tara must never state a figure she did not retrieve.
3. **Handle the "no data" case honestly.** If a question asks about something not in the database, Tara should say so, not invent a number or silently return zero.
4. **Treat tool output as data, not instructions.** Memos are free text written by third parties. Do not let their contents change Tara's behavior.
5. **Validate your tool inputs.** The model is non-deterministic and may call your tools with imperfect arguments. Defensive tools are good tools.
6. **Be deterministic where math is involved.** Totals, rankings, and returns should be computed by code (or by SQL). The model may decide which tool to call, but it should not do arithmetic from raw rows in prose.
7. **Show your formulas and your schema.** In `DESIGN.md`, define your tables, your indexes, your formulas for spend, net spend, merchant matching, recurring detection, fund period return, and holding realised return.
8. **Round consistently.** Two decimal places for currency and percentage answers unless the question asks otherwise.
9. **Add evaluation coverage.** Include a repeatable eval script with questions, expected answers, pass/fail output.
10. **Add basic observability.** Log each task run: question, tools called, sanitized tool inputs, final status, latency, and error reason. Do not log API keys or secrets.
11. **Deploy it.** The deployed service must include a working Postgres connection (managed Postgres on the same host, or a free hosted Postgres like Neon/Supabase/Render Postgres). Free tiers are fine.
12. **You may use AI tools** (Copilot, ChatGPT, Cursor, etc.) freely while building. We assume you will. We are testing the system you ship and your understanding of it, not whether you typed every character yourself.

---

9. Hard cases your solution should support

These are examples, not the full test set.

Spending — single source (transactions table):

- "How much did I spend on food in March 2025 after refunds?"
- "What were my top 5 merchants by net spend between January and March 2025?"

- "Compare my food and travel spending month by month. Which grew faster?"
- "Which category had the biggest increase from February to March?"
- "How much did I spend on Swiggy, including Swiggy Instamart and SWIGGY orders?"
- "Ignore transfers. What was my total actual spending in Q1 2025?"
- "Which transactions look like recurring subscriptions?"
- "Do I have any data for rent in April 2025?"

Funds and holdings — joined sources:

- *Period return (fund only):* "What was Saffron Bluechip Equity Fund's return from 2024-01-01 to 2025-01-01?"
- *Period return ranking:* "Rank all funds by one-year return between 2024-01-01 and 2025-01-01, and show the spread between best and worst."
- *Realised return (holding):* "What is my realised return on my Sentinel Nifty Index Fund holding, given when I bought it?"
- *Portfolio aggregate:* "What is my portfolio worth today, and how much have I made on it in absolute INR?"
- *Mixed:* "Of the funds I own, which gave me the best realised return, and how does it compare to the same fund's period return over the same window?"

A correct answer makes the distinction between "the fund's return between two dates" and "my return on my holding" explicit. We will run some questions more than once — a reliable solution should return the same correct numbers repeatedly.

---

10. Evals, observability, and deployment

We care about whether you can tell if your own agent is working. A good submission should not be a black box.

Evals

Include a small eval suite that we can run from the command line. It should:

- Send multiple questions to your local `/ask` endpoint.
- Compare the answer against expected numbers or expected facts.
- Include at least 12 questions covering single lookup, date filtering, refunds, merchant aliases, transfers, category comparison, recurring subscriptions, no-data cases, fund period returns, and realised returns on holdings.
- Print a clear summary: total passed, total failed, and failed cases.

You can write this as a simple script. You do not need a full testing framework.

Observability

Add logs or a lightweight trace file so we can inspect what happened for each `/ask` request. At minimum, capture:

- `request_id`
- original question
- normalized intent or detected task type, if you compute one
- tools called, in order
- sanitized tool inputs
- which database tables were read
- latency in milliseconds
- success or failure status
- error message or fallback reason, when applicable

Do not log API keys, model secrets, or private environment variables.

Deployment

Deploy the finished service and include the public base URL in your README. You may use Render, Railway, Fly.io, Vercel (serverless or hobby Postgres), AWS, GCP, Azure, or any other host. Free tiers are fine.

The deployed service must:

- Expose `POST /ask` with the same contract as local development.
- Have a working Postgres connection populated by your ingest script.
- Return JSON, not an HTML page.
- Handle a few repeated requests without crashing.
- Document any known limitations of the deployment (cold-start latency, free-tier Postgres limits, etc.).

---

11. What to submit (deliverables)

A single repo (GitHub link or zip) containing:

1. **The code** — your schema, ingest script, tools, agent, orchestration, and the `POST /ask` server. It must run with clear instructions.
2. `README.md` — install / run / deploy instructions, what Postgres setup you used locally, what model/provider, the deployed URL, and any env vars we need.
3. `DESIGN.md` — one to two pages. Explain:
    - Your Postgres schema: tables, columns, indexes, foreign keys, and why.

- Your tool design and why you split them the way you did.
- How you guarantee answers are grounded in the data, not hallucinated.
- Your formulas for spend, net spend, merchant matching, recurring detection, fund period return, and holding realised return.
- How your evals work and what cases they cover.
- What observability you added and how to inspect one failed run.
- Whether you implemented the long-running async tool milestone, and if so, how the agent handles in-progress vs. completed states.
- Where you deployed and any deployment tradeoffs.
- The main ways your system could still fail, and what you'd do with more time.

This document matters a lot. It is how we see your judgment.

4. **Eval script** — repeatable, with questions, expected answers, and pass/fail output.
5. **Observability evidence** — logs, trace output, or screenshots showing at least one successful run and one handled failure/no-data run.
6. **Deployment link** — a public URL where we can call `POST /ask`.

---

12. How we evaluate

We grade on observable behavior, run multiple times for reliability, against a snapshot you have never seen.

Dimension	What we look for
Schema and ingest	Tables make sense, indexes are reasonable, ingest script accepts any snapshot path and produces the same in-database shape.
Grounding	Numbers come from the database via tools. No hallucinated figures, even on unusual values.
Correctness	Aggregates, comparisons, and both fund/holding returns are exactly right.
Multi-step reasoning	Questions needing two or more tool calls (compare, combine, derive, join transactions × holdings × funds) are handled correctly.
Data handling	Refunds, aliases, transfers, date ranges, uncategorized rows, fund/holding distinction handled deliberately. Solution works on all three sample snapshots and on the hidden fourth.
Honesty on edge cases	"No data" is answered honestly. Imperfect inputs are handled, not crashed on.
Reliability	The same question gives a correct answer consistently across repeated runs, not just once.
Async milestone *(optional)*	Slow tools return immediately, work runs in a background worker, agent reasons cleanly across in-progress and completed states, jobs survive a restart.
Evals	Your test harness covers meaningful cases and makes failures visible.
Observability	We can inspect what tools ran, which tables were read, what inputs were used, and why a request failed.
Deployment	The deployed app, including Postgres, is reachable and behaves the same as local development.
Your DESIGN.md	Do you understand and name the system's real tradeoffs and failure modes?

We are not grading on fancy code or features. A simple, correct, well-grounded, reliable agent with a sensible schema and an honest `DESIGN.md` beats a clever, fragile one every time.

---

13. Tips

- **Plan the Postgres + ingest path on day one.** Decide your tables and your ingest contract before you write a single tool. Tools that hardcode column names against the wrong schema are expensive to fix later.
- **Run your ingest against all three sample snapshots early.** If your ingest only works on `sample_a`, you have hidden coupling. Fix it before writing tools.
- **Build a tiny test harness early.** Write a few questions with known answers and run them against your agent as you go. The single most effective thing you can do, and it mirrors how we work.
- **Start simple, then add multi-step.** Get single-lookup questions correct first, then handle comparisons, rankings, month-over-month, and the fund × holdings join.
- **Think about how the model can call your tools wrong**, and make the tools robust to it.
- **Do not overfit** to the sample snapshots in this brief. We test on a fourth one with a different universe. Build something that generalizes.
- **Keep tool outputs lean.** Return what the agent needs to answer, not raw dumps.
- **Prefer SQL/code for math.** Let the model explain the result, not calculate it.
- **Make failures debuggable.** A wrong answer with clear traces is much easier to discuss than a silent black-box failure.
- **Deploy early.** Don't leave deployment for the final hour; most deployment bugs are configuration bugs. Free hosted Postgres (Neon, Supabase, Render) is the fastest path.
- **Decide upfront whether you'll attempt the async milestone.** If yes, design for it on day one; retrofitting is painful.

---

14. Submission

Send us the repo link (or zip), the deployed URL, and anything we need to run it. If you hit a blocker (no model key, deployment issue, a confusing requirement), reach out. Knowing when to ask a good question is a strength, not a weakness.

Good luck. Have fun with it. We are excited to see how you think.